

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

Materia: Base de Datos 2

Fecha: 28 de abril de 2026

Estudiante: Damian Tristant

TP Nro: 3

Tema: Arquitectura, Escalabilidad y Gestores NoSQL

ACTIVIDAD 1: SOLUCIÓN DE INFRAESTRUCTURA (CASO: "TICKET-ALL")

1. Escalabilidad Vertical vs. Horizontal

La propuesta de la empresa de comprar un servidor con más RAM representa una **Escalabilidad Vertical**. Si bien es una solución rápida, tiene un techo técnico y económico: llega un momento en que el hardware no puede crecer más y se convierte en un punto único de falla.

Para este caso, lo ideal es la **Escalabilidad Horizontal**. Al distribuir la carga entre múltiples servidores económicos, no solo eliminamos el cuello de botella del hardware, sino que ganamos redundancia. Si un nodo falla durante una venta masiva, los demás pueden absorber el tráfico.

2. Arquitectura de Sharding y Shard Key

Propongo implementar un cluster particionado. La elección de la **clave de partición** es crítica para evitar puntos calientes.

- **Clave sugerida:** `id_usuario` (usando una estrategia de particionamiento por hash).
- **Justificación:** Si usáramos el ID del evento, todos los compradores de un mismo artista irían al mismo servidor, generando un **punto caliente**. Al usar el ID del usuario o un hash aleatorio, garantizamos que los miles de compradores se repartan equitativamente entre todos los servidores disponibles.

3. Replicación (Replica Sets)

La replicación es la que permite que el sistema no se detenga. En MongoDB, un **conjunto de réplica** se compone de:

- **Nodo Primario:** Es el único que procesa las operaciones de escritura.

- **Nodos Secundarios:** Mantienen una copia exacta de los datos del primario.

Si el Primario falla, el sistema inicia un proceso de elección automática. Uno de los secundarios toma el rol de Primario en segundos, permitiendo que la venta de entradas continúe sin que el usuario final note la caída del servidor original.

ACTIVIDAD 2: MODELADO Y CONSISTENCIA (CASO: "FINTECH-PLUS")

1. ACID vs. BASE

- **Datos A (Salos y transferencias):** Requieren un modelo **ACID** con **Consistencia Inmediata**. En operaciones financieras, el estado del dato debe ser exacto en todo momento para evitar el doble gasto o saldos incorrectos.
- **Datos B (Muro y likes):** Pueden operar bajo el modelo **BASE** con **Consistencia Eventual**. No es crítico que una reacción se vea reflejada instantáneamente para todos los usuarios; priorizamos que el muro cargue rápido.

2. Optimización mediante Desnormalización

En NoSQL, realizar uniones entre colecciones es costoso a nivel de rendimiento. La solución es la **Desnormalización**: guardamos los datos necesarios (como nombre de usuario y foto) directamente dentro del documento del comentario.

Esto optimiza la lectura, ya que con una sola consulta a la base de datos obtenemos toda la información necesaria para mostrar el muro, sin tener que consultar otra colección.

3. Flujo de Consulta

Cuando el usuario abre la aplicación en un entorno distribuido:

1. La consulta llega al **enrutador de consultas**. Este no tiene datos, solo sabe a quién preguntar.
2. El enrutador consulta a los **servidores de configuración**, que actúan como el cerebro del cluster guardando los metadatos y la ubicación de los documentos.
3. Con esa información, el enrutador redirige la petición al servidor específico donde están los datos, recibe la respuesta y la devuelve a la aplicación.

ACTIVIDAD 3: INVESTIGACIÓN

1. Teorema CAP y Principio BASE

El Teorema de Brewer postula que es imposible garantizar simultáneamente Consistencia, Disponibilidad y Tolerancia a Particiones. En sistemas web escalables, la tolerancia a particiones es obligatoria, por lo que debemos elegir entre consistencia o disponibilidad.

El principio **BASE** surge como la alternativa a ACID para sistemas que eligen Disponibilidad sobre Consistencia estricta, permitiendo que el sistema sea escalable y rápido.

Fuente: Documentación técnica sobre sistemas distribuidos.

2. Redis y Tablas de Posiciones

Redis es significativamente más rápido que MongoDB para rankings porque trabaja íntegramente **en memoria RAM**. Además, posee estructuras de datos específicas optimizadas para este propósito.

Comandos investigados:

- INCR: Incrementa de forma atómica el valor de una clave.
- LPUSH: Inserta un elemento al inicio de una lista.
- SADD: Agrega un elemento a un conjunto, garantizando unicidad.

Fuente: Documentación oficial de Redis.

3. Neo4j y Detección de Fraude

Neo4j es una base de datos orientada a grafos. A diferencia de MongoDB, Neo4j trata las relaciones como elementos de primer nivel.

En redes de fraude, la ventaja técnica es su estructura de **Nodos y Aristas**, que permite recorrer conexiones complejas de forma mucho más rápida que mediante consultas anidadas en otras bases de datos.

Fuente: Casos de uso de Neo4j en detección de fraude.